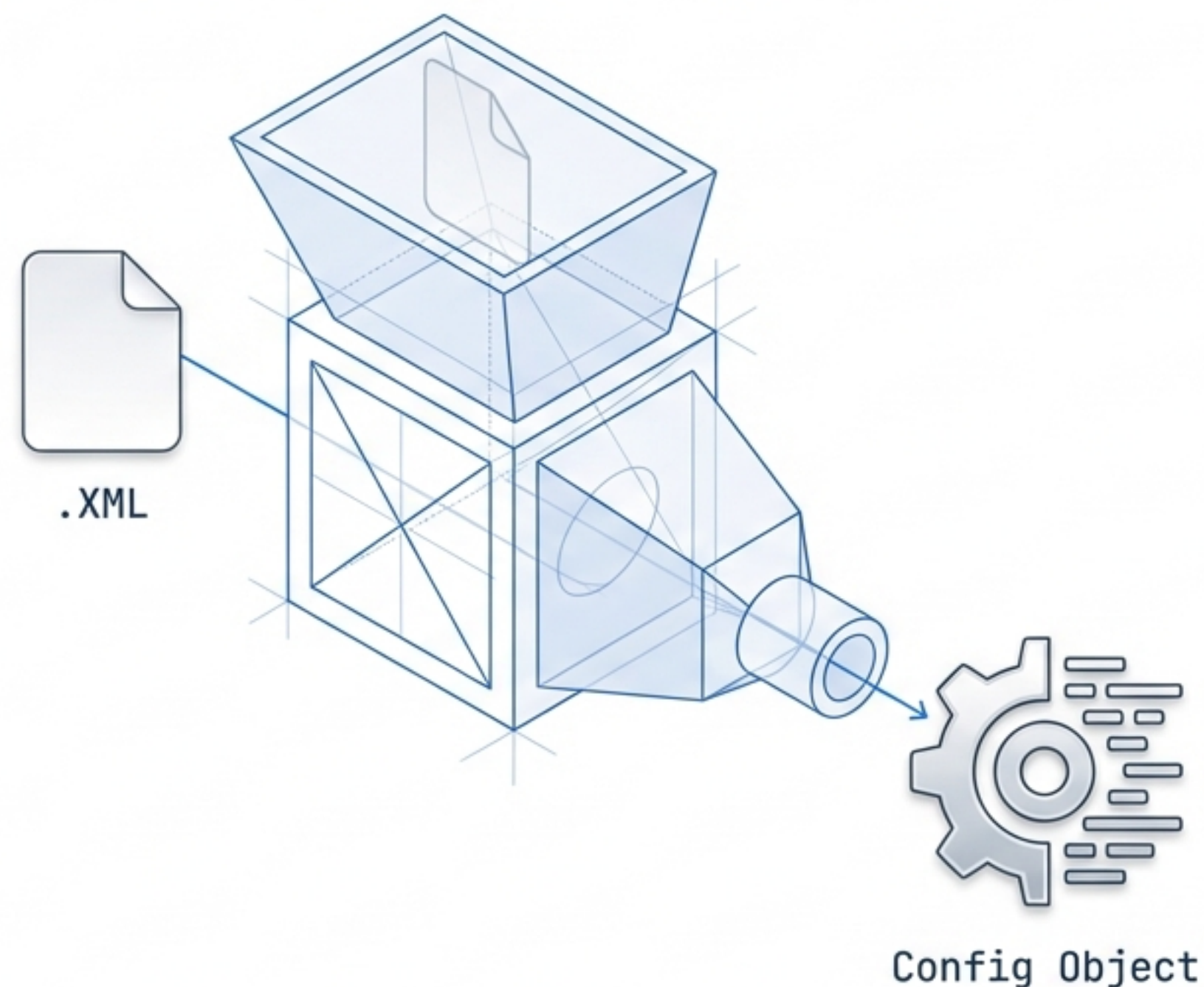


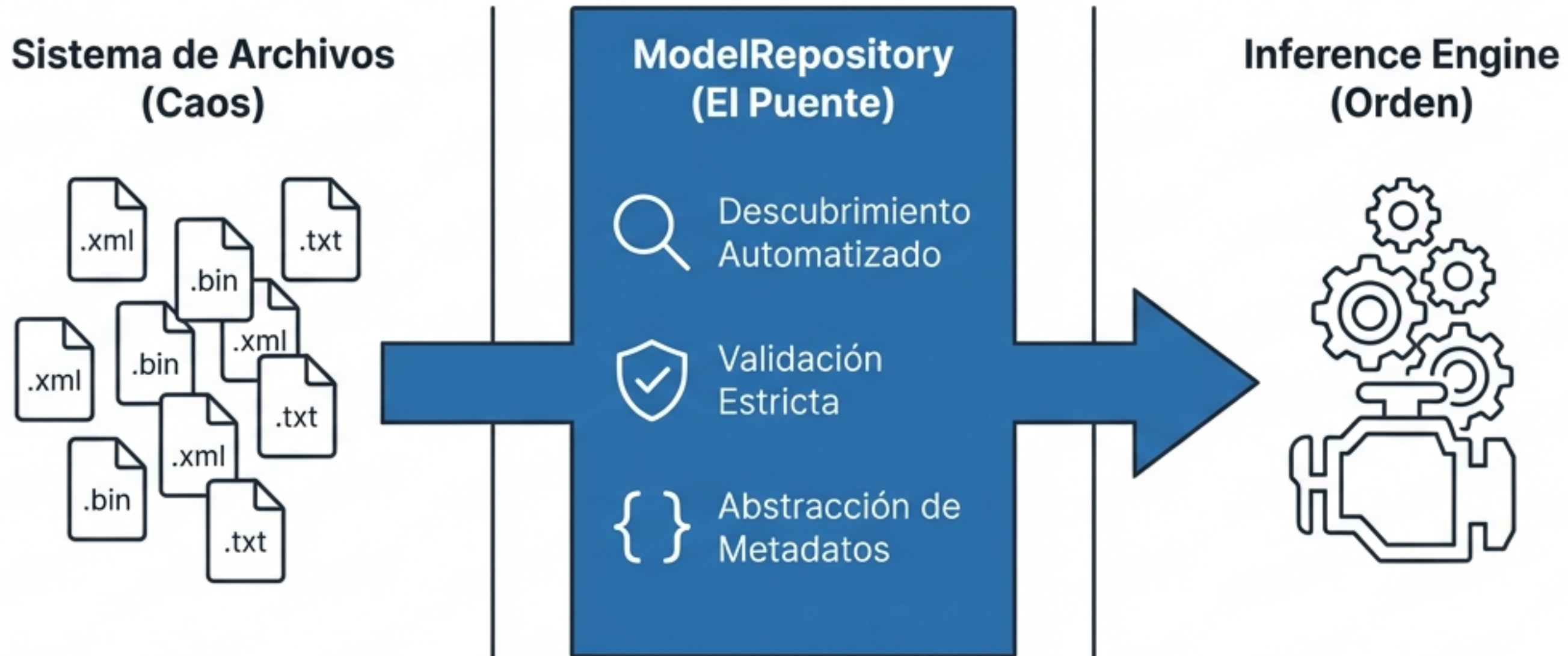
Subsistema de Gestión de Modelos en Bakery-Luna

Arquitectura de Descubrimiento, Validación y Configuración de Modelos OpenVINO



El Puente entre el Disco y la Inferencia

La clase `ModelRepository` actúa como el sistema central de gestión, responsable de transformar archivos estáticos en objetos de configuración listos para la ejecución.



- **Descubrimiento Automatizado:** Escaneo recursivo de directorios.
- **Validación Estricta:** Verificación de formatos YOLO (Seg vs. Pose).
- **Abstracción de Metadatos:** Extracción de shapes y precisión.

El Flujo de Trabajo de Descubrimiento

Proceso secuencial del método `discover_models()`.

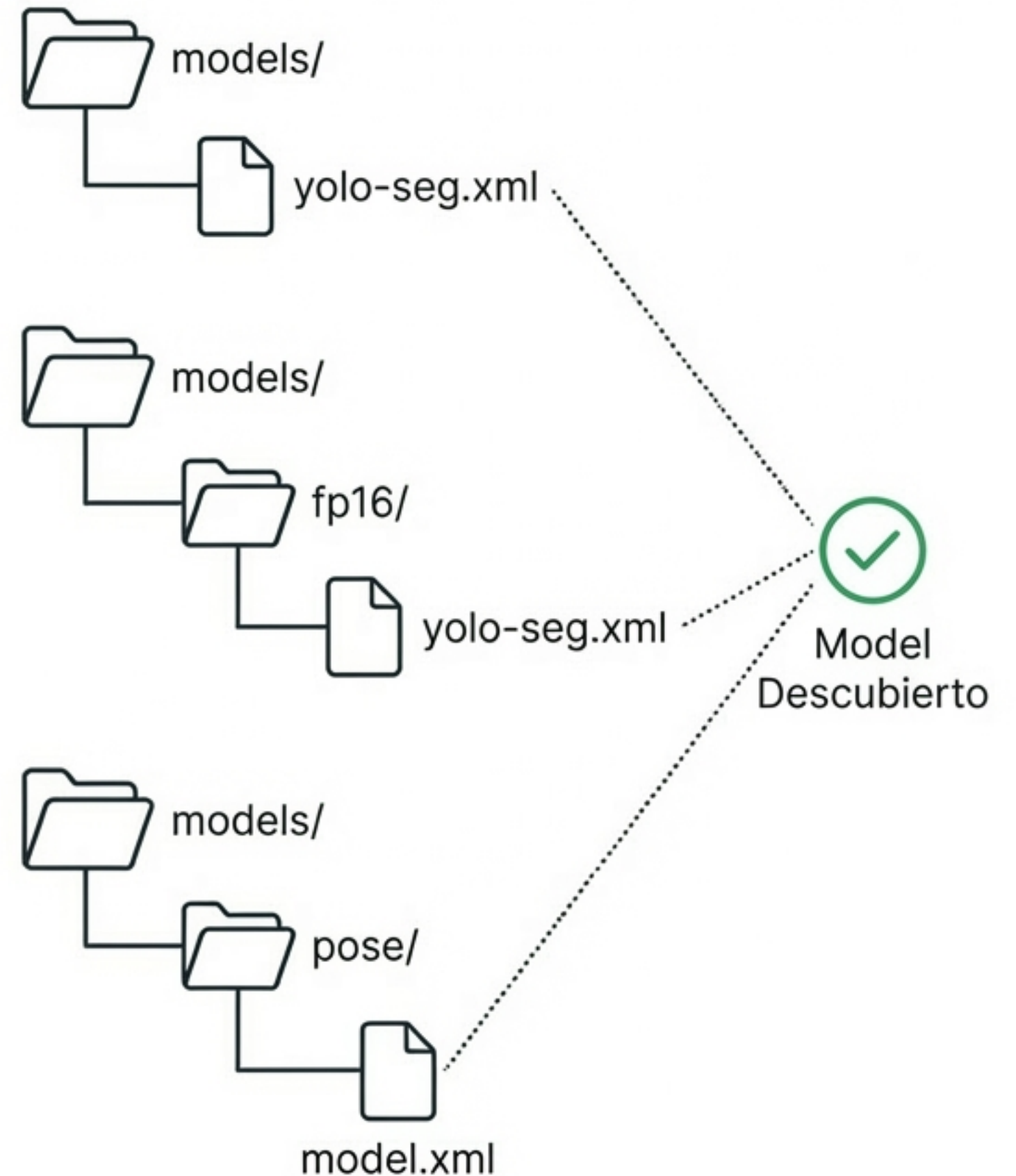


Nota: Si un archivo falla en cualquier etapa, es descartado inmediatamente.
Nota: Si un archivo falla en cualquier etapa, es descartado inmediatamente.

Exploración Recursiva de Directorios

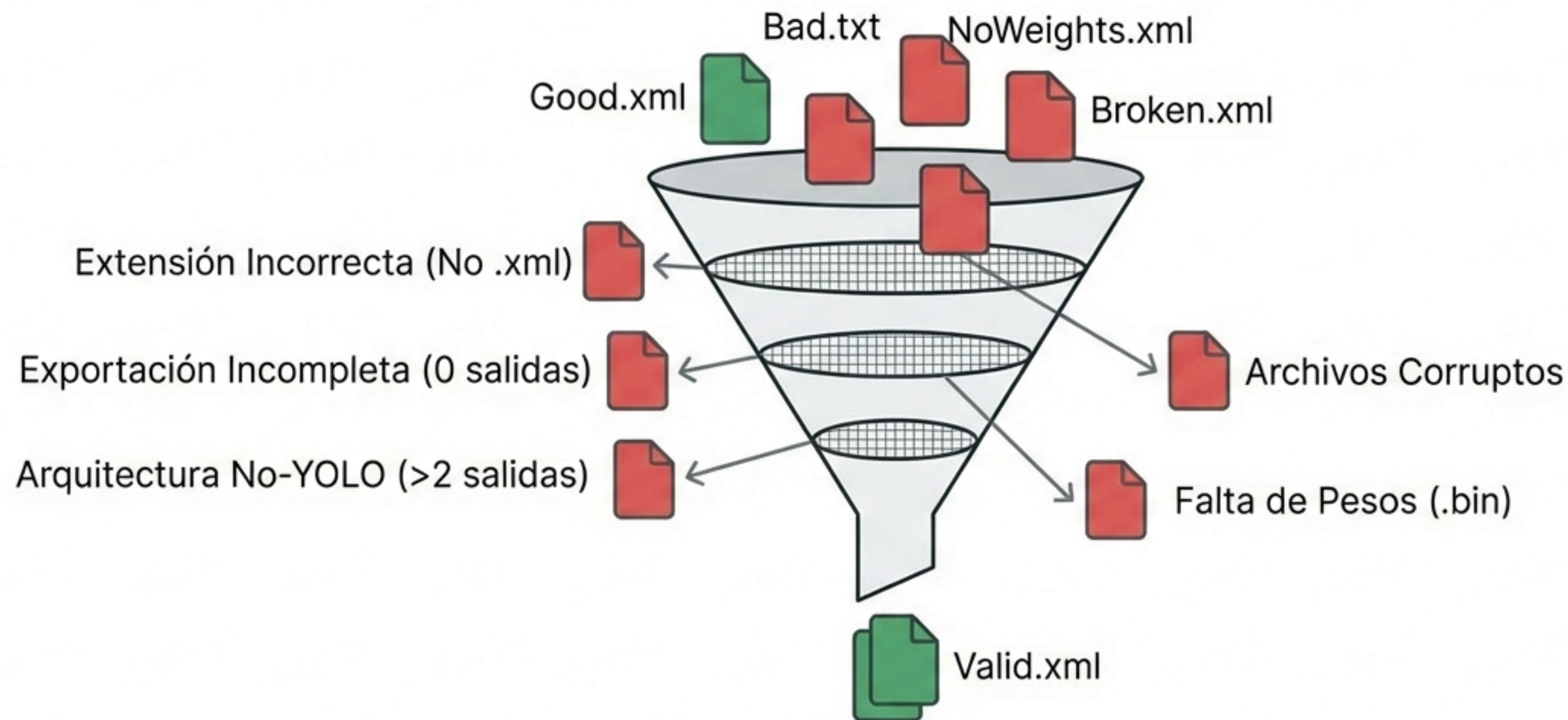
'ModelRepository' soporta estructuras de archivos flexibles, normalizando la ubicación de los modelos independientemente de la profundidad del directorio.

Uso de 'glob' recursivo para evaluación profunda.



El Guardián: Lógica de Validación

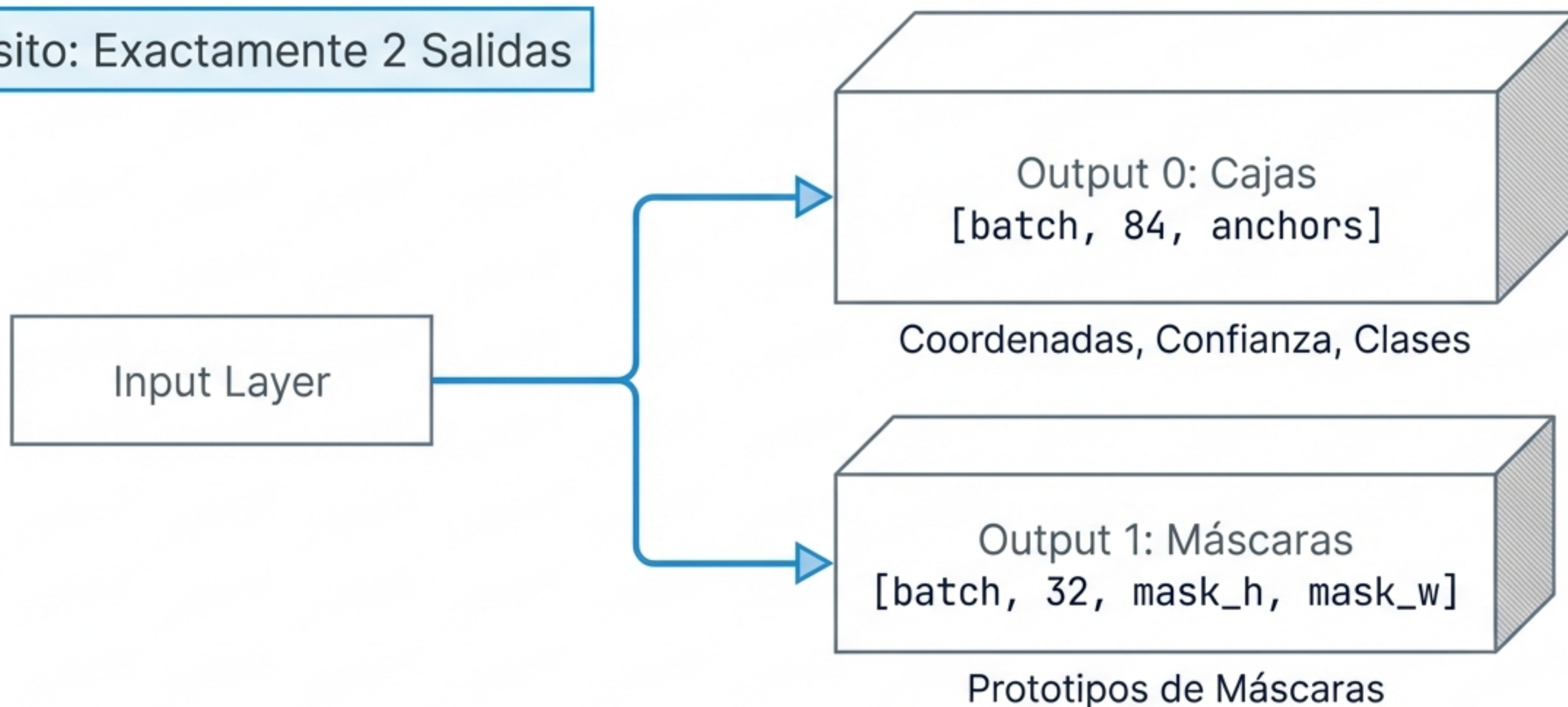
Filtrando modelos inválidos antes de la inferencia.



Estrategia 'Fail Gracefully': Los modelos inválidos se omiten y se registran en el log.

Estándar YOLO: Especificaciones de Segmentación

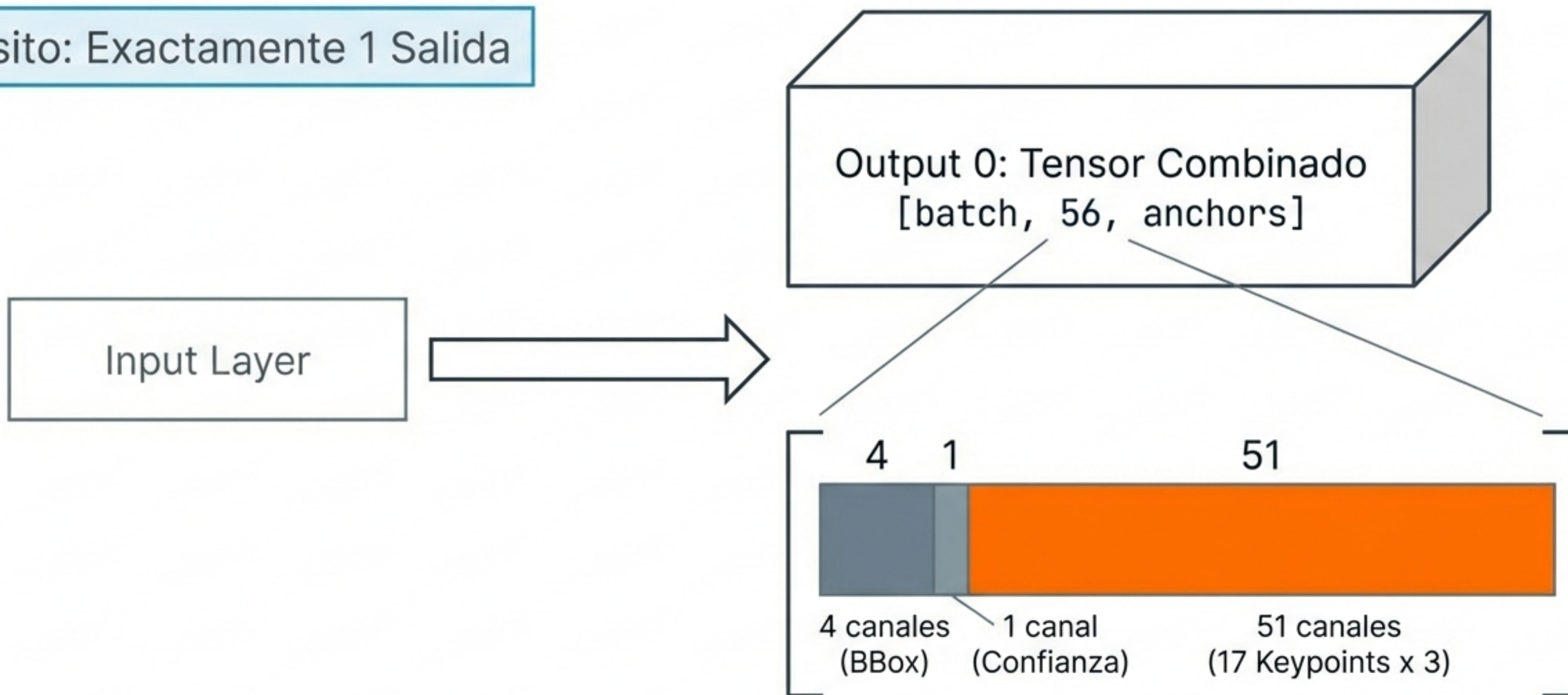
Requisito: Exactamente 2 Salidas



Cualquier desviación en este conteo (1 o 3 salidas) = Modelo Inválido.

Estándar YOLO: Especificaciones de Pose (Keypoints)

Requisito: Exactamente 1 Salida



Detección Automática de Tipo

Clasificación basada en Pattern Matching.

SEGMENTACIÓN



seg_
-seg
_seg
yolo-seg

POSE (Keypoints)

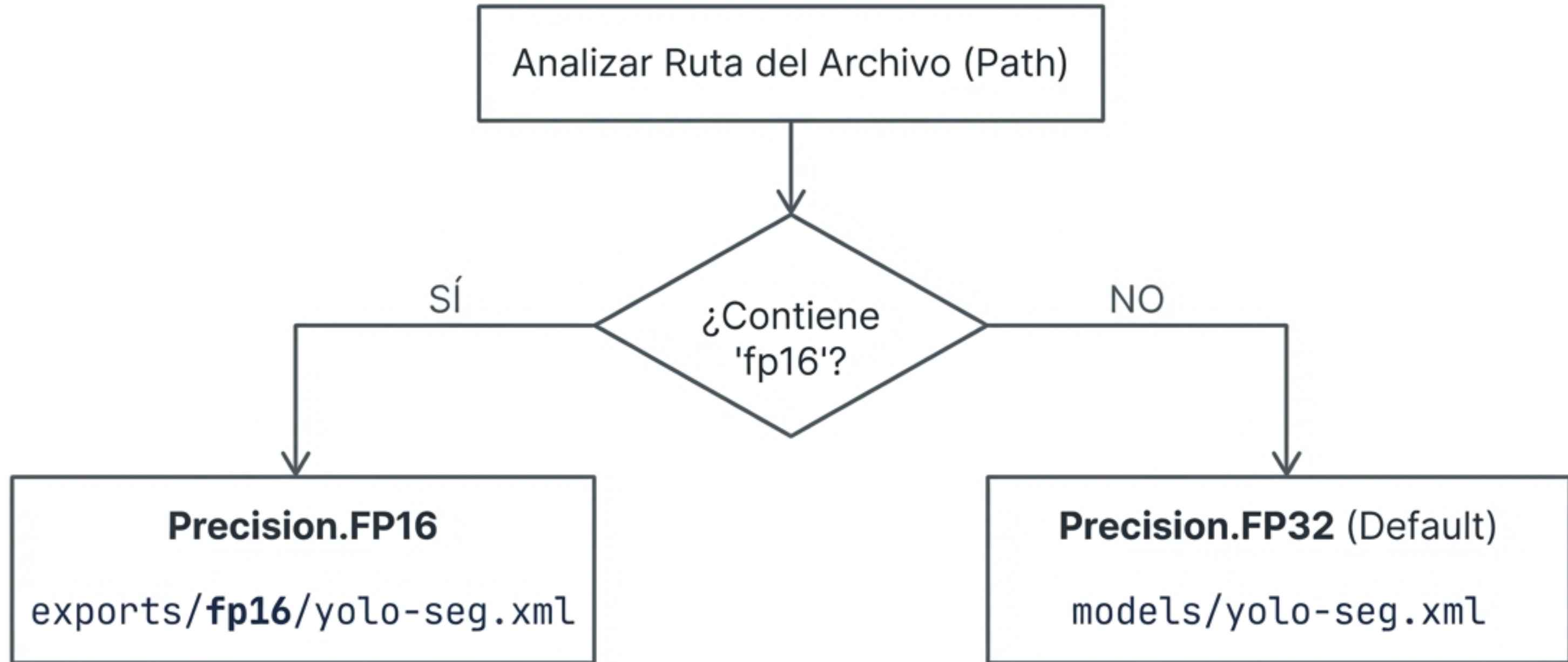


pose_
-pose
_pose
yolo-pose

Fallback de Seguridad: Si no hay coincidencia, el modelo se omite.

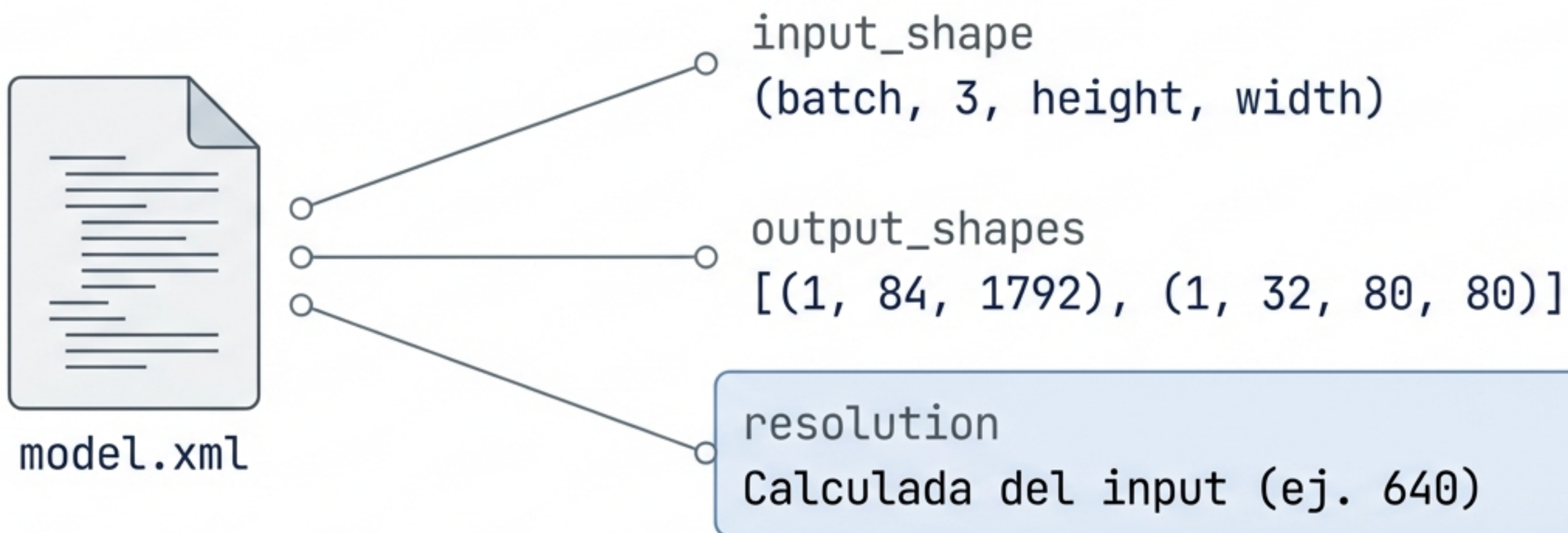
Detección de Precisión (FP16 vs FP32)

La precisión se infiere de la ruta del archivo (path).



Extracción de Metadatos

El método `extract_metadata()` analiza el grafo OpenVINO.



La Resolución es crítica para la optimización. Modelos con igual resolución comparten caché de pre-procesamiento.

La Entidad `ModelConfig`

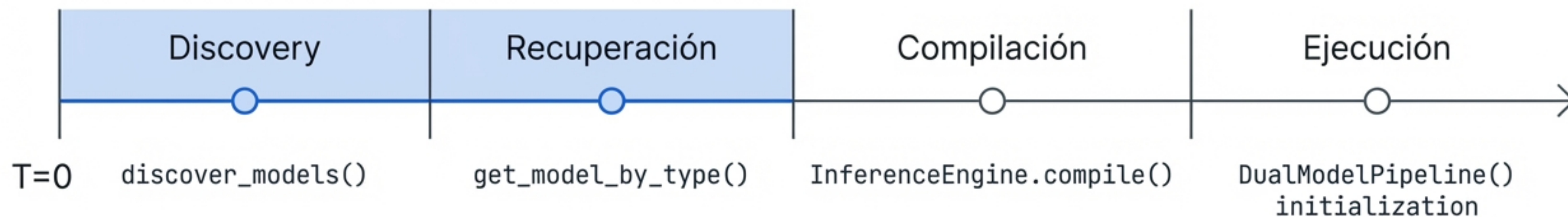
El producto final del proceso de descubrimiento.

```
class ModelConfig
    model_path: "Path(/models/yolo-seg.xml)",
    resolution: "640" (int),
    confidence: "0.65" (float),
    model_type: "ModelType.SEGMENTATION" (Enum),
    precision: "Precision.FP32" (Enum),
    metadata: "{ 'input_shape': ... }" (Dict)
```

Este objeto encapsula todo lo necesario para inicializar el `InferenceEngine`.

Integración en el Pipeline Luna

Secuencia de arranque en `run\_luna.py`



Manejo de Errores y Casos Borde

Fail Gracefully (Descubrimiento) vs. Fail Hard (Inicialización)

Modelo Inválido/Corrupto



SKIP (Log Warning)

Directorio Vacío



RETURN EMPTY

Resultado Lista Vacía



EXIT APP (**Error Crítico**)

Falta un Tipo (Seg/Pose)



ALERT (Pipeline Incompleto)

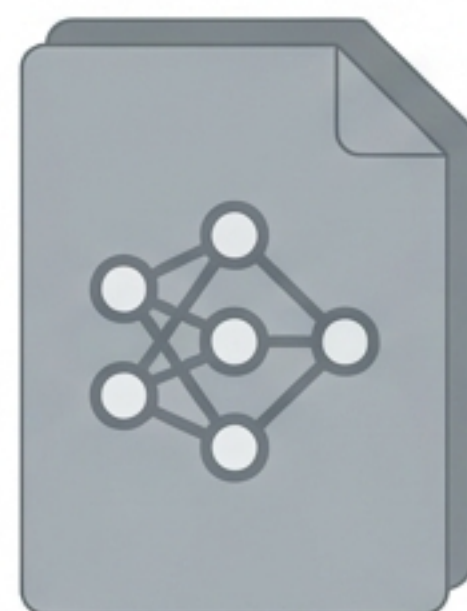
Estrategia de Testing (QA)

Validación sin archivos de peso pesados.

Mocking

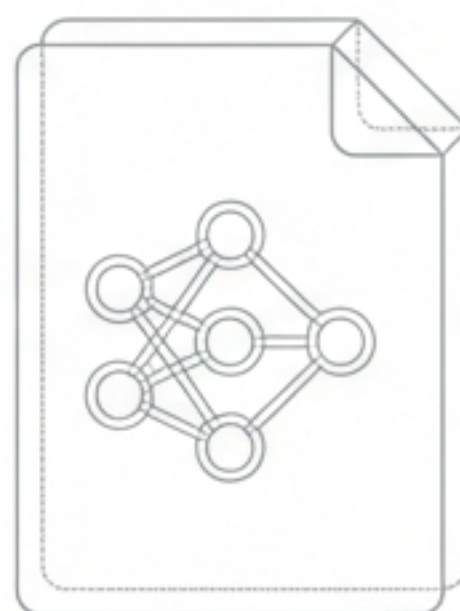
Uso de `openvino.runtime.opset10` para crear operaciones sintéticas.

Mocks imitan inputs y outputs de YOLO.



MODELO REAL

IMITA



MOCK (Sintético)

VALIDA

- ✓ TestModelDiscoveryBehavior (Recursividad)
- ✓ TestModelValidationBehavior (Shapes 2 vs 1)
- ✓ TestMetadataExtractionBehavior

Resumen: Checklist de Requisitos

- ☒ **Archivos:** Par `.xml` y `.bin` con el mismo nombre.
- ☒ **Input:** Capa llamada `images` (float32).
- ☒ **Segmentación:** 2 salidas (Cajas + Máscaras).
- ☒ **Pose:** 1 salida (Tensores combinados).
- ☐ **Nombres:** Incluir 'seg' o 'pose' para detección.
- ☐ **Precisión:** Incluir 'fp16' en la ruta (Opcional).

El `ModelRepository` garantiza que solo modelos compatibles lleguen a producción.